

Présentation : du texte aux données structurées

Dans le module 1 du cours 2, vos agents ont généré du texte au format libre :

```
# Dans les modules précédents
agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Extrais les informations sur les produits à partir des messages des utilisateurs"
)

# Utilisateur : "Je veux l' iPhone 15 Pro avec 256 Go"
# Réponse de l'agent : "L' utilisateur est intéressé par un iPhone 15 Pro avec 256 Go de stockage."
```

Cette approche fonctionne pour les lecteurs humains, mais pose les problèmes suivants pour les systèmes :

-  Aucun format garanti : les réponses varient à chaque exécution
-  Difficulté d'analyse : le traitement du texte est sujet aux erreurs
-  Aucune validation : les champs manquants passent inaperçus
-  Difficulté d'intégration : les bases de données nécessitent une structure cohérente

Dans ce module, nous allons utiliser **output_schema** d'ADK pour appliquer des réponses JSON structurées sur lesquelles les systèmes peuvent se baser.

Le problème : des résultats imprévisibles interrompent les intégrations

Sans sortie structurée, même de bonnes instructions produisent des résultats incohérents :

```
# Aucun schéma de sortie - format imprévisible
text_agent = LlmAgent(
    model="gemini-2.5-flash",
    instruction="Extrais le nom du produit, le prix et la capacité de stockage à partir du message de l'utilisateur"
)

# Que se passe-t-il ?
# Utilisateur : "je veux un iPhone 15 Pro de 256 Go pour 999 $"

# Réponse 1 : "produit : iPhone 15 Pro, prix : 999 $, stockage : 256 Go"
# Réponse 2 : "l'iPhone 15 Pro coûte 999 $ et a 256 Go de stockage"
# Réponse 3 : "iPhone 15 Pro - 999 $ - 256 Go"
```

Le problème principal : votre application ne peut pas analyser de manière fiable ces formats variés. Vous avez besoin d'une structure JSON garantie.